

Custom Merge Sort Report

Reasoning

I implemented a custom merge sort that splits the array into k sublists instead of always splitting it into two halves. In each recursive call, the program asks how many servers are available and uses that value as k . If k is larger than the current array size, I limit it to n .

The array is divided as evenly as possible. Each sublist first receives n / k items, then the remaining items are added one by one to the first few sublists. Each sublist is sorted by calling the same `merge_sort` function again. When a sublist has zero or one item, it is already sorted, so this is the base case. After all sublists are sorted, `merge_sorted_lists` combines them by repeatedly choosing the smallest current value.

Evidence of Testing

The screenshots show five different tests: random order, already sorted, reverse order, duplicate values, and negative values.

```
zsh: command not found: p6
(base) a1234@xiaogedemac C++1008 % ./p6
Enter number of data values:
10
Enter data values:
9 4 7 2 1 6 8 3 5 0
Sorted values:
0 1 2 3 4 5 6 7 8 9
(base) a1234@xiaogedemac C++1008 %
```

Evidence of Testing continued

```
(base) a1234@xiaogedemac C++1008 % ./p6
6
Enter data values:
1 2 3 4 5 6
Sorted values:
1 2 3 4 5 6
• (base) a1234@xiaogedemac C++1008 % g++ p6.cpp -o p6
• (base) a1234@xiaogedemac C++1008 % ./p6
Enter number of data values:
9
Enter data values:
9 8 7 6 5 4 3 2 1
Sorted values:
1 2 3 4 5 6 7 8 9
❖ (base) a1234@xiaogedemac C++1008 %
```

Problems Terminal ... zsh + ▢ 🗑 ...

```
• (base) a1234@xiaogedemac C++1008 % ./p6
Enter number of data values:
7
Enter data values:
5 2 5 1 2 5 3
Sorted values:
1 2 2 3 5 5 5
• (base) a1234@xiaogedemac C++1008 % ./p6
Enter number of data values:
6
Enter data values:
8 -3 0 -10 5 2
Sorted values:
-10 -3 0 2 5 8
```

Time Complexity Analysis

`available_servers()` runs in $O(1)$. Splitting the array copies every item once, so that part is $O(n)$. The main cost is in `merge_sorted_lists()`. The final result has n items, and for each output position my code may check the current element from all k sublists. Therefore one merge step costs $O(nk)$.

If k is fixed during the recursion, the recurrence is:

$$T(n) = kT(n/k) + O(nk)$$

After one split, each problem has size n/k . After two splits, the size is n/k^2 . After L levels, the size is n/k^L . The recursion stops when the size reaches 1, so $n/k^L = 1$. This gives $L = \log_k n$.

Across one level, the total number of items is still n , so the total cost is:

$$O(nk \log_k n)$$

If k is treated as a constant, this simplifies to $O(n \log n)$. However, the scaffold does not let us assume k is constant. In the worst case, k can become n , so the merge may scan n sublists for n output positions. This gives a worst-case complexity of $O(n^2)$. The best behaviour is $O(n \log n)$ when k stays small and constant.

Word Count

This document contains 345 words.